**SAVEETHA SCHOOL OF ENGINEERING**

**OPERATING SYSTEMS**

**ASSIGNMENT | CO1**

# EVALUATE AND JUSTIFY THE OPERATING-SYSTEM DESIGN OF A SMART HOSPITAL

| Student Details | Information |
|---|---|
| Name | S. Maruthu Pandi |
| Register Number | 192511162 |
| Department | B.E. Computer Science and Engineering (Cyber Security) |
| Course | CSA0421 – Operating Systems |
| Assignment | Assignment 1 (Tasks 3) |

## TASK 3 – EVALUATE OPERATING-SYSTEM STRUCTURES

3.1 Main kernel structures

| Structure | How it is organized | Reliability / performance / update trade-off |
|---|---|---|
| Simple / monolithic | Most OS services and drivers execute in kernel space. | Fast system-service paths and low communication overhead, but a faulty driver can corrupt kernel memory and crash the whole system. A large kernel can be harder to isolate and maintain. |
| Layered | OS is arranged in layers; each layer uses lower-level services. | Clear interfaces and easier reasoning/testing. Strict layering may add overhead or restrict efficient cross-layer operations. |
| Microkernel | Kernel contains only essential mechanisms such as scheduling, IPC, basic memory and interrupt handling. Drivers and services run in user space as servers. | Smaller trusted kernel and better fault isolation. Message passing and context switches can add latency; service boundaries make independent updates easier. |
| Modular kernel | A core kernel can load/unload modules such as drivers and filesystems. | Combines a high-performance kernel with flexible updates. A module still executes with kernel privilege, so a faulty module can crash the system. |
| Hybrid | Combines ideas from monolithic and microkernel designs; some services remain in kernel space for performance. | Balances compatibility and performance with some modularity. The exact isolation boundary depends on the implementation. |

3.2 Structure selection for the monitor and database

For the bedside monitor, a small microkernel-style real-time OS is preferred when the chosen platform supports the required drivers and has been validated for the device. The microkernel

keeps scheduling, IPC and interrupt handling in a small trusted core, while sensor and communication drivers can run as isolated services. This reduces the chance that a faulty non-critical driver corrupts the alarm scheduler. The device must still be engineered and tested: a microkernel alone does not prove that a 10 ms deadline will be met.

For the database server, a modular monolithic Linux kernel is a practical choice. Database servers benefit from mature filesystems, networking, storage drivers and efficient system-call paths. Loadable modules support hardware and filesystem needs. Kernel updates and modules must be controlled, tested and patched because a kernel-space module failure can still affect the node.

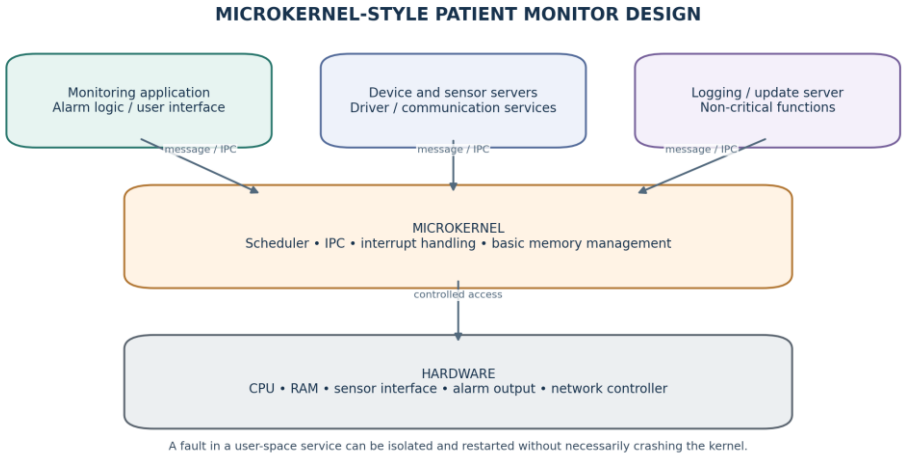



*Figure 2. Microkernel-style monitor: essential scheduling and IPC remain in the kernel while device functions are separated into services.*

3.3 Examples of real operating systems

| Operating system | Broad structural description | Relevance |
|---|---|---|
| Linux | Modular monolithic kernel: core services execute in kernel space and loadable kernel modules provide drivers and other functionality. | Suitable for servers and desktops; a PREEMPT_RT configuration can improve real-time behaviour but must be validated for a specific deadline. |

| Operating system | Broad structural description | Relevance |
|---|---|---|
| Windows NT family | Often described as a hybrid design, with kernel, executive and user-mode subsystems. | Suitable for desktop workstations and enterprise applications. |
| Android | Uses the Linux kernel with Android-specific layers such as hardware abstraction, runtime and application framework. | Suitable for handheld tablets with mobile power and application lifecycle management. |
| FreeRTOS | Small real-time kernel commonly integrated with application code and drivers in an embedded build; it is not a classic user-space-server microkernel. | Suitable for resource-constrained devices when the platform and safety case fit. Driver isolation is more limited than in a microkernel. |
| QNX Neutrino | Microkernel-based RTOS with services and drivers commonly running as separate processes. | A relevant model for safety-related embedded control where isolation and deterministic scheduling are required. |

3.4 Alternative: monolithic kernel in the bedside monitor

In a monolithic design, a request to a kernel service may use one system-call transition and perform the service in kernel space. A microkernel design may send a message to a user-space server, causing IPC and additional scheduling/context-switch work. Therefore, the monolithic path can be faster in raw call overhead. However, the additional microkernel message cost must be measured against the 10 ms deadline and may be acceptable on modern hardware.

A monolithic kernel can have a larger kernel image because more drivers and services are built into privileged space. A microkernel is usually smaller, but total system memory also includes user-space servers, so a smaller kernel does not mean the entire system always uses

less memory. The larger reliability difference is fault containment: a faulty monolithic driver can corrupt privileged memory and crash the OS; a user-space driver in a microkernel may fail independently and be restarted, provided the architecture supports recovery.